aigos

AI-powered fuel trip calculator for Filipino roads

Status	Platform	Language	Developer	Date
Pre-build — Architecture Locked	iOS (SwiftUI)	Swift 6	Solo	April 2026

SECTION 1 — CONTEXT FOR AI: READ THIS FIRST

This document is the **complete, finalized product architecture** for **aigos**. It was produced through a multi-round product strategy session covering every major decision. When a new Claude conversation references this PDF:

Rule 1 — No re-asking	Every decision in this document is FINAL. Do not re-ask questions that are answered here. Treat all choices as locked unless the developer explicitly says otherwise.
Rule 2 — Code mode	Your role is to help the solo developer BUILD the app. Provide full SwiftUI code, not just guidance or snippets. The developer knows React Native (Auxtion marketplace app) but is learning SwiftUI for the first time.
Rule 3 — Start point	Default to Sprint 1 unless the developer specifies a different sprint or task. Sprint 1 = SwiftUI project setup, TabView skeleton, and the core trip calculator screen.
Rule 4 — Offline first	Every feature must work without internet. The only exception is the developer's manual weekly fuel price update. Never suggest cloud solutions.
Rule 5 — iOS only	This is an iOS-only SwiftUI app. No React Native, no cross-platform. Do not suggest Android solutions.

SECTION 2 — INSPIRATION: TARSI BY BRYL KEZTER LIM

Tarsi is a Filipino personal finance app built **solo in 2 days**. It reached #1 on both the App Store and Google Play in the Philippines within 48 hours, earning over **PHP 1 million** in under a month — with zero funding, no backend servers, and no ongoing costs. Its AI feature ('Talk to Tarsi') uses Apple Intelligence on-device for iPhone 15 Pro and 16, with expansion to older devices in v1.1. aigos follows this exact playbook: offline, privacy-first, Filipino-specific, solo-built, one-time price, no backend.

Key lessons from Tarsi applied to aigos: Ship in days not months. No login. No cloud. User controls their data. Solve a Filipino-specific problem global apps ignore. Share revenue milestones publicly on launch day to go viral.

SECTION 3 — PROBLEM STATEMENT

Fuel prices in the Philippines are highly volatile — approximately PHP 62.55/L for gasoline as of April 2026, with 13+ consecutive weeks of increases. Filipino drivers (daily commuters, OFW families on provincial trips, delivery riders, small fleet owners) have **no simple, offline, Filipino-specific tool** to estimate fuel costs before a trip. Global apps are cloud-dependent, require logins, and ignore local context. aigos solves this in one screen, with no login, no cloud, and no ongoing cost.

SECTION 4 — BRAND IDENTITY

App name aigos — lowercase stylized. Never capitalized.	Name origin AI + Agos (Filipino for 'flow' or 'current')
Mascot Smart jeepney — straight-on face with glowing AI eyes. Uniquely Filipino + tech.	Color palette Light & warm — orange/amber tones. Inspired by fuel and SLEX sunsets.
Primary color hex #E9A84C (amber) — used for accents, icons, headers	UI language English only — cleaner and faster to build. No bilingual complexity.
App Store category Travel (primary). Utilities (secondary).	Tagline To be decided after launch. Not a priority for v1.0.

SECTION 5 — TARGET USERS

User Type	Description	Marketing Angle
Daily drivers & commuters	Budget weekly fuel for regular routes (e.g. home to office)	Practical — 'Know your fuel cost before you leave'
OFW families	Plan long provincial drives, calculate round-trip costs	Emotional — 'Plan the drive home before booking the ticket'
Delivery riders	Track fuel as a daily business expense, multiple trips	Functional — 'Know your fuel cost per delivery'
Small fleet owners	Manage multiple vehicle profiles, compare costs	Professional — 'Track every vehicle's fuel spend'

SECTION 6 — STRATEGY SESSION DECISIONS LOG (ALL FINAL)

The following decisions were made during a structured Q&A; session. Each includes the question asked, the answer chosen, and the reasoning. Do not revisit these unless the developer says so.

Q: How will the app get fuel prices?

Decision: User-editable + manual developer update weekly.

Reasoning: Apple Intelligence was considered but is not a price data source — it's a UI layer. Crowdsourcing adds complexity. Scraping DOE is fragile. Manual update by the developer (like Bryl did) is the simplest and most reliable. Users can always override any price, so staleness is always mitigated.

Q: What is the monetization model?

Decision: One-time paid app. PHP 99–199 range.

Reasoning: Matches Tarsi's proven playbook. No subscription friction. Impulse-buy price point. Zero ongoing AI or server costs means every sale is nearly pure margin.

Q: Which platform to launch on first?

Decision: iOS only (SwiftUI). Android deferred.

Reasoning: Originally planned both simultaneously. Switched to iOS-only to use Apple Intelligence natively, avoid cross-platform complexity, and ship faster. Android can come later once the app has revenue and traction.

Q: How to handle AI on iOS vs Android?

Decision: iOS only with Apple Intelligence. Android dropped from scope.

Reasoning: Apple Intelligence requires iPhone 15 Pro / 16 with iOS 18+. Since we went iOS-only, there is no Android AI parity problem to solve.

Q: How to handle offline routing / distance?

Decision: OSRM + preloaded Philippine OpenStreetMap dataset. Fully offline.

Reasoning: Google Maps Distance API requires internet and has per-call costs that scale with users. OSRM is open-source, can be bundled in the app, works offline, and has zero per-call cost. Manual distance input is the fallback if the bundle proves too large (>100MB).

Q: How to protect App Store rating from price inaccuracy?

Decision: Disclaimer on all results + user-editable price field.

Reasoning: Users who feel in control of the input data won't blame the app for wrong outputs. Disclaimer sets expectations. Editable prices give agency.

Q: What language for the UI?

Decision: English only.

Reasoning: Faster to build. Cleaner UI. Filipino tech-savvy users are comfortable in English. Bilingual adds string management complexity for a solo developer.

Q: Which fuel efficiency unit?

Decision: km/L default. L/100km available as a toggle in Settings.

Reasoning: km/L is the standard in the Philippines for car specs and everyday conversation. L/100km is available for imported or international spec vehicles.

Q: What is the app navigation structure?

Decision: SwiftUI TabView with 5 tabs. Flat navigation, no deep stacks.

Reasoning: Same pattern Tarsi uses. Every feature is one tap away from any screen. Simplest possible navigation for a solo developer to build and maintain.

Q: What is the AI strategy for v1.0 vs v1.1?

Decision: v1.0: Apple Intelligence on iPhone 15 Pro and 16 only. v1.1: expand to older devices.

Reasoning: This is Bryl Lim's exact Tarsi playbook. Ship the AI feature for supported devices first to reduce launch complexity. Expand later using iOS 17+ on-device NLP alternatives via Foundation Models framework.

Q: Should the app have a mascot?

Decision: Yes — a smart jeepney with glowing AI eyes, straight-on face.

Reasoning: Tarsi's tarsier mascot was critical to virality and social sharing. A jeepney is uniquely Filipino, universally recognizable, and unexpected as an app mascot. The glowing AI eyes tie in the tech angle without being generic.

Q: What is the primary launch channel?

Decision: All three simultaneously: TikTok, Facebook (OFW/driver groups), and Threads/X.

Reasoning: Tarsi went viral partly because Bryl publicly shared his revenue milestone. TikTok is for app demo reach. Facebook is where Filipino OFW and driver communities live. Threads/X is for the developer story angle that gets shared in tech circles.

Q: How does the user tell the app about their driving style?

Decision: Talk to aigos hybrid interview — chat bubble personality + tap-to-answer buttons.

Reasoning: Voice-first is too clunky pre-drive. Full chat requires typing. The hybrid gives aigos personality (mascot chat bubble) while keeping answers to a single tap — fast, offline-friendly, no keyboard needed.

Q: How accurate should fuel estimates be?

Decision: Physics-based — factors in speed, AC use, and road type per trip.

Reasoning: Simple multipliers (smooth/aggressive) feel arbitrary to users. Physics-based factors (speed curve, AC penalty, road type) are explainable and produce a breakdown that builds user trust in the estimate.

Q: When does the driving style interview happen?

Decision: Per trip + self-corrects over time from trip history.

Reasoning: One-time setup gets stale. Per-trip keeps it accurate and surfaces the feature repeatedly. Self-learning means the app gets smarter the more you use it — a strong retention mechanic.

Q: Which physics factors are included in v1.0?

Decision: Speed (highway + city avg), AC usage, and road type (highway/city/mountain).

Reasoning: These three factors are the most significant and most understandable to Filipino users. Speed is the biggest variable in fuel consumption. AC is nearly universal in PH. Road type captures EDSA vs NLEX vs Kennon Road differences clearly.

Q: How should the adjusted result be displayed?

Decision: Full breakdown: distance, base litres, each adjustment factor with % and amount, total.

Reasoning: A breakdown makes the estimate feel honest and explainable. Users who understand why the number is what it is will trust and share the app. Tarsi's success came from transparency — aigos should do the same.

Q: How is driving style scoped — per vehicle or global?

Decision: Per vehicle — each vehicle has its own DrivingProfile linked via SwiftData relationship.

Reasoning: Driving a Vios is different from driving a truck. City bike different from highway car. Per-vehicle profiles make the estimates significantly more accurate and encourage users to add all their vehicles.

Q: Who is building this?

Decision: Solo developer. Same as Bryl Lim with Tarsi.

Reasoning: Zero coordination overhead. Ship fast. Every decision is made instantly. The entire build should be completable in 6 days of focused work.

SECTION 7 — CORE ARCHITECTURE DECISIONS (SUMMARY)

Fuel price source	User-editable + manual developer update weekly via app update or bundled JSON file. Last-updated date always displayed. All prices editable before calculating.
Routing / distance	OSRM + preloaded Philippine OpenStreetMap dataset bundled in app. Fully offline. No API cost. Manual distance input as fallback if bundle >100MB.
AI / voice input	Apple Intelligence — Foundation Models framework. On-device, zero API cost, zero latency, works offline. iPhone 15 Pro + 16 at v1.0. Expand to older iPhones in v1.1.
Data persistence	SwiftData — local only. No CoreData. No UserDefaults for complex objects. No cloud sync. No iCloud. Everything stays on device.
Fuel units	km/L default. L/100km toggle in Settings.
Accuracy protection	Disclaimer on all results. User-editable fuel prices. Last-updated date shown.
Platform	iOS only. SwiftUI + Swift 6. Minimum iOS 17. Android deferred.
Backend	NONE. Zero servers. Zero cloud. Zero ongoing cost.
Monetization	One-time paid app. PHP 99–199. Apple App Store only.

SECTION 8 — CORE CALCULATION FORMULA

This is the exact math the app performs. Implement this precisely.

```
// ■■ Base Inputs ■■■
```

```
// 1. Speed factor — optimal fuel economy is around 80-90 km/h.
//    Above or below, efficiency drops.
let avgHighwaySpeedKmh: Double // user-reported, e.g. 100
let avgCitySpeedKmh: Double    // user-reported, e.g. 35
let speedFactor: Double = calculateSpeedFactor(highway: avgHighwaySpeedKmh,
                                               city: avgCitySpeedKmh)
// speedFactor examples: 60 km/h city = 1.08 (+8%), 120 km/h hwy = 1.15 (+15%)

// 2. AC factor — aircon increases fuel use 10-15% in PH conditions
let acInUse: Bool              // user-reported per trip
let acFactor: Double = acInUse ? 1.12 : 1.0  // +12% when AC on

// 3. Road type factor — uphill/mountain burns significantly more fuel
let roadType: RoadType         // .highway | .city | .mountain
let roadFactor: Double = switch roadType {
    case .highway:  1.00   // baseline
    case .city:     1.20   // stop-and-go +20%
    case .mountain: 1.35   // elevation changes +35%
}
// enum RoadType: String, Codable { case highway, city, mountain }

// ■■ Combined Adjustment Multiplier ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
let adjustmentMultiplier: Double = speedFactor * acFactor * roadFactor
// Example: highway, 110 km/h, AC on = 1.13 * 1.12 * 1.00 = 1.27 (+27%)

// ■■ Adjusted Efficiency ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
let adjustedEfficiencyKmL: Double = baseEfficiencyKmL / adjustmentMultiplier

// ■■ Final Calculation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
let litresNeeded: Double  = distanceKm / adjustedEfficiencyKmL
let estimatedCost: Double = litresNeeded * fuelPricePerLitre

// ■■ Result Breakdown (shown to user) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Distance:            82.4 km
// Base litres:         5.7 L  (at 14.5 km/L rated)
// Speed adjustment:   +13%  (avg 110 km/h highway)
// AC adjustment:      +12%  (aircon on)
// Road adjustment:    +0%   (highway)
// Adjusted litres:     7.3 L
// Estimated cost:      PHP 456.15
// [Disclaimer shown below]

// ■■ L/100km mode (Settings toggle) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
let kmLfromL100: Double = 100.0 / l100kmValue  // convert before using above
```

Important: Use PHP as the currency prefix (not the peso sign character) to ensure correct rendering across all iOS devices and PDF exports.

SECTION 9 — SWIFTDATA SCHEMA (DATA MODELS)

All models use @Model from SwiftData. No CoreData. No UserDefaults for complex data. Store in the default ModelContainer.

VehicleProfile (updated — now links to DrivingProfile)

```
@Model class VehicleProfile {
    var id: UUID = UUID()
    var name: String           // e.g. 'My Vios', 'Delivery Bike'
    var type: VehicleType      // enum: .car | .motorcycle | .truck | .van
    var fuelType: FuelType     // enum: .ron91 | .ron95 | .ron97 | .diesel
    var efficiencyKmL: Double  // manufacturer km/L rating, e.g. 14.5
    var isDefault: Bool = false
    var createdAt: Date = Date()
    @Relationship var drivingProfile: DrivingProfile?  // per-vehicle profile
}

enum VehicleType: String, Codable { case car, motorcycle, truck, van }
enum FuelType: String, Codable { case ron91, ron95, ron97, diesel }
enum RoadType: String, Codable { case highway, city, mountain }
```

TripRecord (auto-saved on every calculation, includes driving factors)

```
@Model class TripRecord {
    var id: UUID = UUID()
    var origin: String
    var destination: String
    var distanceKm: Double
    var baseLitres: Double         // litres at manufacturer rating
    var adjustedLitres: Double     // litres after driving profile factors
    var speedFactor: Double        // e.g. 1.13
    var acFactor: Double           // 1.12 or 1.0
    var roadFactor: Double         // 1.00, 1.20, or 1.35
    var fuelPriceAtTime: Double    // snapshot when calculated
    var totalCostPHP: Double
    var vehicleName: String        // snapshot
    var fuelType: FuelType
    var avgHighwaySpeed: Double
    var avgCitySpeed: Double
```

```
    var roadType: RoadType
    var acWasOn: Bool
    var calculatedAt: Date = Date()
    var isFavorite: Bool = false
}
```

DrivingProfile (NEW — per vehicle, learned over time)

```
@Model class DrivingProfile {
    var id: UUID = UUID()
    var vehicleId: UUID

    // User-reported (set via Talk to aigos interview)
    var avgHighwaySpeedKmh: Double = 90.0   // default: 90 km/h
    var avgCitySpeedKmh: Double = 40.0       // default: 40 km/h
    var usesACByDefault: Bool = true         // default: true (PH climate)
    var defaultRoadType: RoadType = .highway

    // Self-correcting fields (updated after each trip)
    var tripCount: Int = 0
    var avgActualHighwaySpeed: Double?   // learned from history
    var avgActualCitySpeed: Double?      // learned from history
    var acUsageRate: Double = 1.0        // % of trips with AC on

    // Confidence score — low until 5+ trips logged
    var confidenceLevel: ProfileConfidence = .new
    var lastUpdated: Date = Date()
}

enum ProfileConfidence: String, Codable {
    case new        // 0-4 trips: show 'Learning your style...'
    case building   // 5-14 trips: show 'Getting to know you'
    case confident  // 15+ trips: show 'Based on your history'
}
```

SavedRoute (starred from History or manually saved)

```
@Model class SavedRoute {
    var id: UUID = UUID()
    var name: String           // user-given label e.g. 'Home to Office'
    var origin: String
    var destination: String
    var distanceKm: Double
    var defaultVehicleId: UUID? // optional preferred vehicle for this route
```

```
    var savedAt: Date = Date()
}
```

FuelPriceCache (one row per fuel type)

```
@Model class FuelPriceCache {
    var id: UUID = UUID()
    var fuelType: FuelType
    var pricePHP: Double        // e.g. 62.55
    var lastUpdated: Date
    var updatedBy: String = 'developer' // 'developer' | 'user'
}

// Seed on first launch with approximate current prices:
// RON 91: 59.10  |  RON 95: 62.55  |  RON 97: 65.20  |  Diesel: 56.45
// User can override any price in the Prices tab at any time.
```

SECTION 10 — FULL FEATURE LIST V1.0

#	Feature	Description
01	Trip calculator	Enter origin + destination (OSRM autocomplete). Select vehicle. Physics-based calculation using driving
02	Smart Driving Profile	Per-vehicle profile storing avg highway speed, city speed, AC usage, and road type preference. Set via T
03	Fuel price panel	Prices tab: RON 91, RON 95, RON 97, Diesel. Last-updated date shown. All prices user-editable. Develo
04	Vehicle profiles	Add/edit/delete vehicles with name, type, fuel type, km/L rating, and linked DrivingProfile. Supports: car,
05	Saved routes	Star any trip to save as a named route. One-tap recalculate with today's prices and current driving profile
06	Trip history	Every calculation auto-saved with full breakdown snapshot: speed factor, AC factor, road factor, adjusted
07	Talk to aigos (AI interview + 2 more)	TWO modes: (1) Pre-trip interview — hybrid chat bubble + tap-to-answer buttons ask about speed, AC, ro
08	Settings	Fuel unit toggle (km/L / L/100km). Default vehicle. Reset driving profile. Clear history. Price update prefe

---

SECTION 11 — NAVIGATION STRUCTURE

SwiftUI TabView with 5 tabs. Flat navigation — no deep NavigationStack hierarchies. Every feature is one tap from any screen. This is the same pattern Tarsi uses.

Tab	Icon (SF Symbol)	Screen	Purpose
1 (Default)	car.fill	Calculate	Main trip calculator
2	clock.fill	History	Auto-saved trip log with comparisons
3	star.fill	Saved Routes	Starred routes, one-tap recalculate
4	fuelpump.fill	Prices	Editable fuel prices per type

5	gearshape.fill	Settings	Units, defaults, clear history

```
// ContentView.swift — TabView skeleton
struct ContentView: View {
    var body: some View {
        TabView {
            CalculateView()
                .tabItem { Label('Calculate', systemImage: 'car.fill') }
            HistoryView()
                .tabItem { Label('History', systemImage: 'clock.fill') }
            SavedRoutesView()
                .tabItem { Label('Saved', systemImage: 'star.fill') }
            PricesView()
                .tabItem { Label('Prices', systemImage: 'fuelpump.fill') }
            SettingsView()
                .tabItem { Label('Settings', systemImage: 'gearshape.fill') }
        }
        .accentColor(Color(hex: '#E9A84C'))
    }
}
```

SECTION 12 — ONBOARDING FLOW (4 SCREENS)

Shown only on first launch. Minimal and fast — goal is first calculation in under 90 seconds. No account, no email, no cloud.

Screen	Content	CTA
Screen 1 — Welcome	aigos mascot (jeepney with AI eyes). App name. 'Know your exact fuel cost before every trip.'	'Get started'
Screen 2 — Add your vehicle	Name, type (car/moto/truck/van), fuel type, km/L efficiency. Pre-filled example: 'My Car, RON 95, 12 km/L'.	'Save vehicle'
Screen 3 — Your driving style	aigos mascot chat bubble: 'Let me learn how you drive so I can give you smarter estimate.' 3 tap-to-answer questi	'Looks good'
Screen 4 — Fuel prices	Current pre-loaded prices shown. 'Updated weekly. You can always edit before calculating.'	'Start calculating'

SECTION 13 — APPLE INTELLIGENCE: GATE LOGIC & IMPLEMENTATION

Talk to aigos uses the Foundation Models framework introduced in iOS 18.1. The feature must be gated by device capability at runtime.

```
// Gate check — run before showing Talk to aigos UI
import FoundationModels

func isAppleIntelligenceAvailable() -> Bool {
```

```
    // Foundation Models availability check
    if #available(iOS 18.1, *) {
        return LanguageModelSession.isSupported  // true on A17 Pro+ devices
    }
    return false
}

// In Talk to aigos view:
if isAppleIntelligenceAvailable() {
    TalkToAigosView()  // full AI chat UI
} else {
    // Show friendly message:
    // 'Talk to aigos requires iPhone 15 Pro or iPhone 16.
    //  Full calculator available below.'
    CalculateView()    // fallback to standard UI
}
```

v1.0: Gate on A17 Pro chip (iPhone 15 Pro, iPhone 16 series). **v1.1:** Evaluate Foundation Models availability on older A15/A16 devices as Apple expands support. Expand accordingly.

SECTION 14 — SMART DRIVING PROFILE: FULL SPECIFICATION

This is aigos's key differentiator. No other fuel calculator app in the Philippines adjusts estimates based on how YOU drive. This section covers the full flow — from interview to calculation to self-learning.

14A. The Pre-Trip Interview (Talk to aigos — Hybrid Mode)

When a user taps Calculate, aigos optionally runs a quick interview. UX: aigos mascot chat bubble appears with a friendly message, then tap-to-answer buttons appear below each question. No typing. No keyboard. Fast and offline.

Question	Tap Options	Maps To
How fast do you usually drive on the highway?	Under 80 km/h / 80-100 km/h / 100-120 km/h / Over 120 km/h	avgHighwaySpeedKmh: 70 / 90 / 110 / 130
How about in the city or traffic?	Slow (under 30) / Normal (30-50) / Moving fast (50+)	avgCitySpeedKmh: 25 / 40 / 55
Will you be using aircon?	Yes, always on / Sometimes / No	acInUse: true / true (0.5 weight) / false
What kind of road is most of your trip?	Highway (NLEX, SLEX) / City roads / Mountain / Mix	tripType: .highway / .city / .mountain / weighted avg

14B. Per-Trip Override vs Saved Profile

aigos remembers the last answer for each vehicle. Before each trip, it shows: 'Same as last time? (Highway, 100-120 km/h, AC on)' with a quick confirm button. User can confirm in one tap or change any factor. This makes repeat trips instant while still allowing full customization.

14C. Self-Learning Engine

After each trip, the DrivingProfile updates using a rolling average. The confidence level is shown on the result screen.

```
// ProfileLearningEngine.swift
func updateProfile(_ profile: DrivingProfile, with trip: TripRecord) {
    profile.tripCount += 1

    // Rolling average — weight recent trips more (last 10 trips matter most)
    let weight = min(Double(profile.tripCount), 10.0)
    profile.avgActualHighwaySpeed = rollingAvg(
        existing: profile.avgActualHighwaySpeed ?? trip.avgHighwaySpeed,
        new: trip.avgHighwaySpeed,
        weight: weight
    )
    profile.avgActualCitySpeed = rollingAvg(
        existing: profile.avgActualCitySpeed ?? trip.avgCitySpeed,
        new: trip.avgCitySpeed,
        weight: weight
    )

    // AC usage rate
    let acCount = Double(profile.tripCount)
    profile.acUsageRate = ((profile.acUsageRate * (acCount - 1))
        + (trip.acWasOn ? 1.0 : 0.0)) / acCount

    // Update confidence level
    profile.confidenceLevel = switch profile.tripCount {
        case 0..<5:  .new
        case 5..<15: .building
        default:     .confident
    }
    profile.lastUpdated = Date()
}

func rollingAvg(existing: Double, new: Double, weight: Double) -> Double {
    return ((existing * (weight - 1)) + new) / weight
}
```

14D. Result Breakdown Display

The result screen shows a full breakdown — not just a number. This builds trust and explains the estimate.

Line	Example	Notes
Distance	82.4 km	From OSRM or manual

Base fuel (rated)	5.7 L	At manufacturer km/L
Speed adjustment	+13% (0.8 L)	Based on avg 110 km/h hwy
AC adjustment	+12% (0.7 L)	Aircon on
Road adjustment	+0% (0.0 L)	Highway selected
Adjusted fuel	7.2 L	Total litres estimated
Fuel price	PHP 62.55 / L	Editable — tap to change
ESTIMATED COST	PHP 450.36	Bold, large, amber color
Confidence	Based on 12 trips	ProfileConfidence label
Disclaimer	Estimate only. Tap any value to adjust.	Always shown

14E. Speed Factor Lookup Table

Reference values used in calculateSpeedFactor():

Avg Speed	Speed Factor	Reason
Under 60 km/h (heavy city)	1.18	Stop-and-go, idling, frequent braking
60-80 km/h (light city/suburb)	1.08	Moderate efficiency loss
80-100 km/h (optimal highway)	1.00	Peak fuel efficiency zone
100-120 km/h (fast highway)	1.13	Aerodynamic drag increases
Over 120 km/h (very fast)	1.22	Significant drag and fuel waste

---

SECTION 15 — FUEL PRICE UPDATE STRATEGY

DOE (Department of Energy) Philippines publishes pump price updates every Tuesday. The developer updates aigos prices weekly using this process:

Method	How It Works	Effort
App update (v1.0)	Bundle a FuelPrices.json file in the app. Each weekly price update = update JSON + submit new app version to App Store. Users get it when they update the app.	Medium — App Store review takes 1–3 days
Remote JSON (v1.1 upgrade)	Host a simple static JSON file (e.g. GitHub raw or Cloudflare Pages — free). App fetches on launch, caches locally. Developer updates JSON only — no app update needed.	Low — edit one file, done in 5 minutes
User override (always available)	User can tap any price in the Prices tab and edit it directly. This is the ultimate safety net — users are never blocked by stale data.	Zero — already in the app

Launch with app-update method. Migrate to remote JSON in v1.1.

SECTION 16 — TECH STACK

Component	Choice	Notes
Language	Swift 6	Latest stable Swift
UI Framework	SwiftUI	iOS 17+ minimum target. Native, deep Apple Intelligence integration.

AI / NLP	Apple Foundation Models	On-device. Free. Zero API calls. Gate on iOS 18.1 + A17 Pro.
Routing	OSRM + OpenStreetMap PH	Bundled in app. Fully offline. Manual distance fallback if >100MB.
Local Storage	SwiftData	Trip history, saved routes, vehicle profiles, fuel price cache.
Map display	None in v1.0	App calculates distance only. No map view needed.
Backend / Cloud	None	Zero servers. Zero ongoing cost. Privacy-first.
Analytics	None	No tracking. No analytics SDK. Privacy-first.
Distribution	Apple App Store	iOS only. One-time paid. PHP 99–199.
IDE	Xcode 16+	Required for SwiftUI + Foundation Models development.

SECTION 17 — XCODE PROJECT STRUCTURE

```
aigos/
  aigos.xcodeproj
  aigos/
    App/
      aigosApp.swift          // @main entry point, ModelContainer setup
      ContentView.swift       // TabView with 5 tabs
    Models/
      VehicleProfile.swift    // @Model SwiftData entity
      TripRecord.swift        // @Model SwiftData entity
      SavedRoute.swift        // @Model SwiftData entity
      FuelPriceCache.swift    // @Model SwiftData entity
      Enums.swift             // VehicleType, FuelType enums
    Views/
      Calculate/
        CalculateView.swift   // Main trip calculator
        RouteInputView.swift  // Origin/destination text fields
        VehiclePicker.swift   // Vehicle selection sheet
        ResultCard.swift      // Cost result + disclaimer
      History/
        HistoryView.swift     // Trip log list
        TripRowView.swift     // Individual trip row
      SavedRoutes/
        SavedRoutesView.swift // Saved routes list
        SavedRouteRow.swift   // Individual saved route row
      Prices/
        PricesView.swift      // Fuel price panel
```

```
      PriceRowView.swift    // Editable price row per fuel type
    Settings/
      SettingsView.swift    // Settings screen
    TalkToAigos/
      TalkToAigosView.swift // Apple Intelligence chat UI
      AIGateView.swift      // Shown when device not supported
    Onboarding/
      OnboardingView.swift  // 3-screen onboarding flow
  Services/
    CalculationEngine.swift // Core fuel cost formula
    OSRMService.swift       // Offline routing wrapper
    PriceService.swift      // Fuel price read/write logic
  Resources/
    FuelPrices.json         // Default prices, updated weekly
    PH_roads.osrm           // Philippines road dataset (compressed)
  Assets.xcassets           // App icon, colors, mascot image
```

SECTION 18 — APP STORE LISTING DETAILS

App name	aigos
Subtitle	Fuel trip cost calculator
Category	Travel (primary) / Utilities (secondary)
Price	PHP 99 at launch. Can raise to PHP 199 after initial traction.
Keywords	fuel cost, gas calculator, road trip, petrol, gasoline, Philippines, OFW, commute, diesel, trip planner
Description hook	Before you drive, know what you'll spend. aigos calculates your exact fuel cost from any point A to point B — offline, in seconds, no sign-up needed.
Screenshots needed	1) Calculate screen showing Manila-to-Batangas result. 2) History screen with comparison. 3) Saved routes. 4) Prices tab. 5) Talk to aigos (AI chat).
Privacy URL	Not needed — app collects zero data.
Support URL	Developer's social media or a simple landing page.

SECTION 19 — KNOWN RISKS & MITIGATIONS

HIGH	**Fuel prices go stale**	Show last-updated date on Prices tab. User can edit any price. Disclaimer on all results. Developer updates weekly via app update (v1.0) or remote JSON (v1.1).
MEDIUM	**OSRM bundle size**	Compressed PH-only road tiles targeted under 100MB. If over limit, use App Store on-demand resources or fall back to manual distance input.
MEDIUM	**OSRM road accuracy**	OpenStreetMap PH is well-maintained. Show 'estimated distance' label. Manual distance input always available as override.
MEDIUM	**Apple Intelligence device gate**	Core app works 100% without AI on all iPhones. AI is enhancement only. Show clear message on unsupported devices. v1.1 expands support.
LOW	**Bad early App Store reviews**	Disclaimer + editable prices sets user expectations. Soft launch to small group (family, friends, driver FB groups) before going public.
LOW	**Solo developer burnout**	6-day sprint with clear milestones. Follow Tarsi's playbook: ship imperfect, iterate fast. Feature creep is the enemy.

SECTION 20 — BUILD & LAUNCH ROADMAP

SPRINT 1 — Core calculator (Days 1–2)

[ ] Xcode project setup with SwiftData ModelContainer [ ] TabView with 5 tabs and placeholder views [ ] VehicleProfile, TripRecord, SavedRoute, FuelPriceCache SwiftData models [ ] OSRM integration PROTOTYPE (most critical risk — do this first) [ ] CalculateView: origin/destination input, vehicle picker, result card [ ] CalculationEngine.swift with core formula [ ] PricesView: editable fuel prices, last-updated date

SPRINT 2 — Memory & polish (Days 3–4)

[ ] Auto-save every calculation to TripRecord [ ] HistoryView: list of trips, month-over-month comparison [ ] SavedRoutesView: star to save, one-tap recalculate [ ] SettingsView: units toggle, default vehicle, clear history [ ] OnboardingView: 3-screen first-launch flow [ ] App icon design: jeepney face, glowing AI eyes, amber palette [ ] Mascot asset (used in onboarding and Talk to aigos)

SPRINT 3 — AI & App Store prep (Days 5–6)

[ ] Apple Intelligence integration via Foundation Models framework [ ] TalkToAigosView with natural language trip queries [ ] AIGateView for unsupported devices [ ] App Store screenshots (5 required) [ ] App Store description and keywords [ ] Final QA: test offline mode, all 5 tabs, onboarding, AI gate [ ] Submit for App Review

LAUNCH DAY

[ ] TikTok: 60-second demo video — calculate Manila to Batangas in real time [ ] Facebook: post in OFW groups, car enthusiast groups, driver communities [ ] Threads/X: developer story post — announce the app and the build story [ ] Post revenue milestone updates as they happen (Tarsi playbook) [ ] All three channels simultaneously

v1.1 — Post-launch (Week 2+)

[ ] Migrate fuel prices to remote JSON (GitHub/Cloudflare Pages) [ ] Expand Talk to aigos to older iPhones (iOS 17+) [ ] Ship improvements based on first-week user feedback [ ] Evaluate Android port based on revenue and demand

---

SECTION 21 — LAUNCH CHANNELS

TikTok	Demo video: calculate a real Manila-to-Batangas trip. Show result in under 10 seconds. Caption: 'Built this in 6 days. No login, no cloud, works offline.'
Facebook	Post in: OFW communities, Pinoy car enthusiast groups, provincial driver groups, motorcycle rider groups, delivery rider groups.
Threads/X	Developer story: share the build process, the sprint, and first revenue number. Same angle Bryl Lim used that made Tarsi go viral in tech circles.
Timing	All three channels simultaneously on launch day for maximum momentum. Follow up with revenue milestone posts (PHP 10k, PHP 50k, PHP 100k).

SECTION 22 — NOTES FOR THE AI BUILDING PARTNER

1. Full code always: The developer knows React Native but is learning SwiftUI. Always provide complete, copy-pasteable SwiftUI code — not pseudocode, not snippets, not guidance only.

2. Prototype OSRM first: The OSRM integration (offline routing with bundled PH road data) is the highest technical risk. Build and test this before anything else in Sprint 1.

3. SwiftData strictly: Use SwiftData for all persistence. No CoreData. No UserDefaults for complex objects. No iCloud sync. No cloud at all.

4. Apple Intelligence gate: Always gate Talk to aigos with isAppleIntelligenceAvailable(). The core app must work perfectly without AI on any iPhone running iOS 17+.

5. Offline first: Every feature must work without internet. The only exception is the optional remote JSON price fetch (v1.1). Never make a core feature internet-dependent.

6. Peso display: Display currency as 'PHP' prefix (e.g. 'PHP 261.43') not the peso sign character, to ensure correct rendering on all devices.

7. Tarsi philosophy: When in doubt: simple, fast, private, Filipino-first, no login, no cloud. Ship imperfect. Iterate fast. The goal is a working app in 6 days.

8. Sprint 1 definition of done: A user can: select a vehicle, enter origin and destination, tap calculate, and see an estimated fuel cost in PHP. Everything else is Sprint 2 and beyond.